

Control de altura por realimentación negativa

Motor Dron TP — Sistemas Embebidos de Posgrado

Evolución de un banco de pruebas de motor DC + sensor ultrasónico hacia un sistema de control en lazo cerrado con telemetría inalámbrica

Plataforma STM32F446RE (Nucleo-F446RE)	IDE / HAL STM32CubeIDE · HAL STMicroelectronics
Interfaz remota Node-RED · Bluetooth HC-05 (SPP)	Tipo de control Lazo cerrado proporcional ($K_p = 18 \text{ \%/cm}$)

Punto de partida: El proyecto original

El firmware inicial consistía en un **único main.c** que combinaba dos experimentos de laboratorio independientes, sin ningún lazo de control entre ellos.

Experimento 1 — Control de velocidad motor DC

- ADC1 en **PA4 (canal 4)** con filtro de promedio móvil de 16 muestras
- PWM de **1 kHz** generado por TIM2_CH1 / PA5 → driver L298N (ENA)
- Sentido fijo adelante: IN1/IN2 en PB5/PB10
- Velocidad controlada directamente por el potenciómetro analógico

Experimento 2 — Medición de distancia HC-SR04

- TIM1 en modo **Input Capture**: TRIG en PA9, ECHO en PA8/TIM1_CH1
- Cálculo de ancho de pulso → distancia en cm
- Reporte por **USART2 a 115200 baud** al ST-Link (terminal serie)
- Frecuencia de muestreo: ~1 Hz

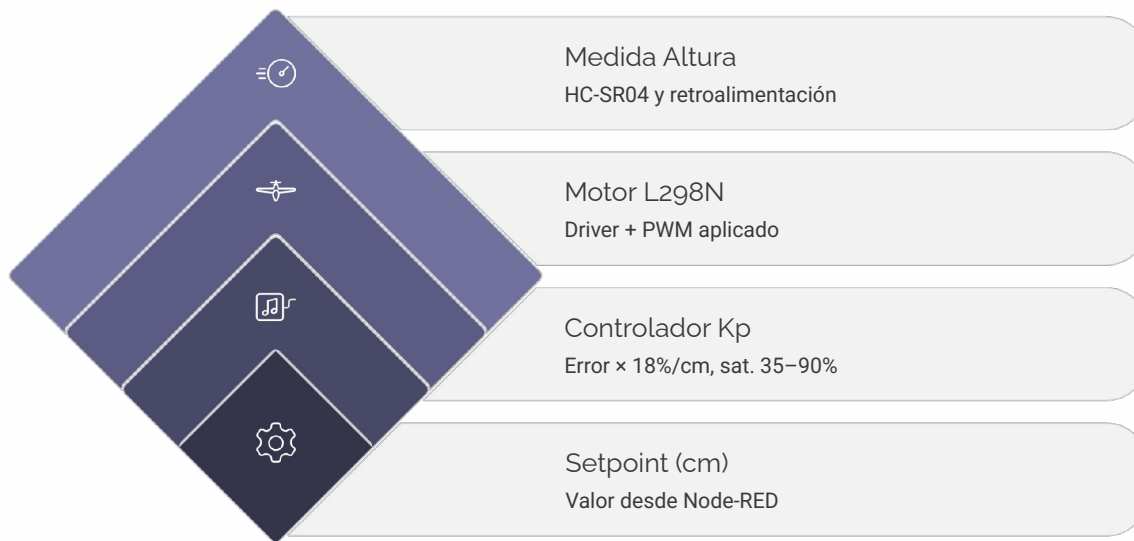
❌ **Punto crítico:** Motor y sensor NO estaban conectados entre sí. La distancia se medía pero no influía en la velocidad. No existía lazo de control, Bluetooth ni Node-RED.

Objetivo del rediseño

Transformar el banco de pruebas en un **verdadero sistema de control automático en lazo cerrado**, eliminando la entrada manual del potenciómetro y habilitando operación remota.

- 1 Retroalimentación real**
La distancia medida por el HC-SR04 se convierte en la variable de retroalimentación que ajusta automáticamente velocidad y sentido del motor en cada ciclo de control.
- 2 Setpoint remoto**
La altura objetivo (setpoint en cm) es configurable de forma remota por Bluetooth desde un dashboard Node-RED, sin recompilar ni reconectar el hardware.
- 3 Telemetría continua**
El sistema emite telemetría a 2 Hz (altura, setpoint, velocidad aplicada) hacia Node-RED para visualización en tiempo real y análisis de respuesta al escalón.

Diagrama de bloques — Lazo de realimentación negativa



El error se calcula como $e = \text{setpoint} - \text{altura_actual}$. Una zona muerta de ± 0.5 cm evita chattering por ruido de medición. El lazo opera a **10 Hz**; la telemetría se emite a 2 Hz de forma desacoplada.

Arquitectura de software: La modularidad

La reorganización completa en módulos independientes (carpetas `Src/` e `Inc/`) es la decisión arquitectónica central del rediseño. Cada módulo tiene **responsabilidad única**, es testeable de forma aislada y reemplazable sin afectar al resto.



motor.c / motor.h

Driver L298N. Expone `Motor_Aplicar(sentido, velocidad_pct)` con enum `MOTOR_DETENER / MOTOR_SUBIR / MOTOR_BAJAR`. Oculta PWM (TIM2_CH1/PA5) y pines de dirección (PB5/PB10).



distance.c / distance.h

Driver HC-SR04 con Input Capture (TIM1). Filtro de promedio móvil circular de 8 muestras. Expone: `Distancia_Actualizar()`, `Distancia_ObtenerCm()`, `Distancia_UltimaFueValida()`.



height_control.c / height_control.h

Controlador proporcional puro. **No conoce hardware**. Expone: `ControlAltura_FijarObjetivoCm()`, `ControlAltura_Actualizar()`, getters de telemetría.



hc10.c / hc10.h

Driver Bluetooth (UART4). Recepción por interrupción, parseo de `HEIGHT:<valor>`, envío de telemetría `H: , SP: , V:` a 2 Hz.



main.c — Orquestador

Solo inicializa periféricos y ejecuta el superloop. No contiene lógica de control ni de hardware. Estilo de registros crudos HAL/CMSIS por coherencia con lo aprendido.

Tabla de conexionado físico (pines)

Asignación completa de pines del Nucleo-F446RE para el sistema integrado. **GND común obligatorio** entre fuente de motor y STM32 (falla real detectada y corregida en puesta en marcha).

Función	Pin Nucleo	Periférico / Notas	Subsistema
ENA (PWM)	PA5	TIM2_CH1, 1 kHz, duty cycle 35–90%	Motor L298N
IN1 (dirección)	PB5	GPIO salida digital	Motor L298N
IN2 (dirección)	PB10	GPIO salida digital	Motor L298N
TRIG	PA9	GPIO salida, pulso 10 µs	HC-SR04
ECHO	PA8	TIM1_CH1, Input Capture	HC-SR04
TX módulo BT	PA1	UART4_RX, 9600 baud	Bluetooth HC-05
RX módulo BT	PA0	UART4_TX, 9600 baud	Bluetooth HC-05
TX debug	PA2	USART2, 115200 baud (ST-Link)	Debug
RX debug	PA3	USART2, 115200 baud (ST-Link)	Debug

 USART2 y UART4 son periféricos separados deliberadamente para evitar contención entre tráfico de debug (115200 baud) y tráfico Bluetooth (9600 baud). Nunca comparten bus.

Sistema de control: Lazo de realimentación negativa

Ley de control proporcional implementada en `height_control1.c`. El controlador opera a **10 Hz** (cada 100 ms), frecuencia aumentada desde 1 Hz original para reducir overshoot.

Parámetros del controlador

18

Kp (%/cm)

Ganancia proporcional

± 0.5

Zona muerta (cm)

Evita chattering

10

Hz lazo

Frecuencia de control

Saturación mínima

35% – vence fricción e inercia

Saturación máxima

90% – margen de seguridad

Rango válido

3 a 20 cm (HC-SR04 confiable)

Pseudocódigo de la ley de control

```
error = objetivo - altura_actual

if |error| < 0.5 cm:
    motor → DETENER
    velocidad = 0

else if error > 0:
    sentido = MOTOR_SUBIR
    vel = min(Kp × error, 90%)
    vel = max(vel, 35%)
    motor → SUBIR @ vel

else:
    sentido = MOTOR_BAJAR
    vel = min(Kp × |error|, 90%)
    vel = max(vel, 35%)
    motor → BAJAR @ vel
```

El motor permanece **detenido hasta recibir el primer setpoint válido** por Bluetooth (bandera `objetivo_recibido`), garantizando arranque seguro.

Comunicación Bluetooth: Protocolo y hardware

Enlace Bluetooth completo agregado al sistema (no existía en el proyecto original). Implementado sobre **UART4 (PA0/PA1) a 9600 baud**. El hardware real identificado es un **HC-05** (breakout ZS-040, Bluetooth clásico/SPP), no HM-10 (BLE).

Protocolo de entrada — Comando remoto desde Node-RED

```
HEIGHT:<valor_cm>\n
```

Ejemplo: `HEIGHT:15\n` fija el setpoint a 15 cm. El módulo `hc10.c` recibe por interrupción y parsea el token `HEIGHT:` antes de llamar a

```
ControlAltura.FijarObjetivoCm().
```

Protocolo de salida — Telemetría a 2 Hz

```
H:<altura>,SP:<setpoint>,V:<velocidad_pct>\n
```

Ejemplo: `H:12.5,SP:15,V:45\n` — formato CSV simple, fácil de parsear en Node-RED. Desacoplado del lazo de 10 Hz.

Roadmap — Protocolo extendido (futuro)

```
H:,SP:,V:,VZ:,S:,L:\n
```

Agrega velocidad vertical (VZ), validez de sensor (S) y flag de lazo (L), pensado para futuro término derivativo Kd. El parser de Node-RED ya está preparado.

Puesta en marcha del módulo HC-05

Identificación del hardware

El módulo real es HC-05 (SPP), no HM-10 (BLE). Diferencia crítica: pin EN/KEY activa modo comandos AT a **38400 baud** vs. modo datos a **9600 baud**.

Firmware puente standalone

Herramienta de soporte: firmware que usa el ST-Link como adaptador USB-TTL para enviar comandos AT al HC-05, con barrido automático de baudrates para identificar configuración de fábrica.

Separación de código

Los firmwares de diagnóstico (puente AT, adaptación BLE/HC-05) se mantienen en variantes aisladas, **nunca mezclados** con el código de producción del sistema de control.

 USART2 emite simultáneamente mensajes legibles por humanos a 115200 baud para depuración en terminal serie, en paralelo al tráfico Bluetooth.

Dashboard en Node-RED: Interfaz de usuario remota

Flujo control-altura-dashboard.json — HMI/SCADA liviano que reemplaza la terminal serie cruda, permite ensayos sin recompilar y separa la visualización de la lógica de control embebida.

Grupo: Consigna

- Slider de **3 a 20 cm** para ajuste continuo del setpoint
- Botones de acceso rápido: escalón **5 cm** → **15 cm** para ensayos de identificación de lazo

Grupo: Respuesta al escalón

- Gráfico de línea: **altura medida vs. setpoint** en el tiempo
- Visualización de overshoot, tiempo de establecimiento y error en estado estacionario

Grupo: Velocidad vertical

- Gauge de **empuje aplicado (%)** en tiempo real
- Gráfico de velocidad vertical del motor

Nodo function — "Formatear HEIGHT"

```
// Arma string HEIGHT:<n>\n
// Valida rango 3-20 cm
// Replica hacia dos salidas:
//   → Puerto serie USB/ST-Link
//   → Puerto Bluetooth HC-05
// Sin cambiar firmware embebido
```

Nodo function — "Parsear telemetría"

```
// Interpreta línea recibida:
// H:12.5,SP:15,V:45\n
// Extrae: altura, setpoint, velocidad
// Alimenta gráficos y gauges
// Parser preparado para protocolo
// extendido: VZ, S, L (futuro Kd)
```


Fragmentos de código: Módulo motor.c

Driver del puente H L298N con **responsabilidad única**: abstraer completamente el hardware de actuación. El resto del sistema solo llama `Motor_Aplicar()`. Cambiar el driver (MOSFET, servo, ESC) requiere editar únicamente este módulo.

Interfaz pública — motor.h

```
typedef enum {
    MOTOR_DETENER,
    MOTOR_SUBIR,
    MOTOR_BAJAR
} motor_senso_t;

void Motor_Aplicar(
    motor_senso_t sentido,
    uint8_t velocidad_pct
);
```

Implementación — motor.c

```
void Motor_Aplicar(motor_senso_t sentido,
                   uint8_t velocidad_pct) {
    // Saturación de entrada
    velocidad_pct = (velocidad_pct > 100)
        ? 100 : velocidad_pct;

    // CCR para PWM 1 kHz (ARR = 1000)
    uint32_t ccr = (velocidad_pct * 1000) / 100;
    TIM2->CCR1 = ccr; // PA5 (ENA)

    switch(senso_t) {
        case MOTOR_SUBIR:
            GPIOB->ODR |= (1 << 5); // IN1=1
            GPIOB->ODR &= ~(1 << 10); // IN2=0
            break;
        case MOTOR_BAJAR:
            GPIOB->ODR &= ~(1 << 5); // IN1=0
            GPIOB->ODR |= (1 << 10); // IN2=1
            break;
        case MOTOR_DETENER:
            TIM2->CCR1 = 0;
            break;
    }
}
```

Acceso directo a registros

TIM2->CCR1 y GPIOB->ODR en lugar de funciones HAL de alto nivel, coherente con el estilo HAL/CMSIS del proyecto.

Encapsulamiento total

PA5, PB5, PB10 y TIM2 son detalles internos. Ningún otro módulo conoce estos pines.

Extensibilidad

Reemplazar L298N por un ESC o MOSFET requiere solo reescribir este archivo, sin tocar `height_control.c` ni `main.c`.

Fragmentos de código: Módulo height_control.c

Controlador proporcional puro **sin conocimiento de hardware**. Testeable sin MCU real. Cambiar Kp o zona muerta no toca motor.c ni distance.c.

Interfaz pública — height_control.h

```
void ControlAltura_FijarObjetivoCm(
    float objetivo);
void ControlAltura_Actualizar(
    float altura_actual);
float ControlAltura_ObtenerError(void);
uint8_t ControlAltura_ObtenerVelocidad(void);
```

Constantes y estado interno

```
#define KP          18.0f // %/cm
#define ZONA_MUERTA 0.5f // cm
#define VEL_MIN      35 // %
#define VEL_MAX      90 // %

static float objetivo_cm    = 0.0f;
static float error_cm       = 0.0f;
static uint8_t vel_aplicada = 0;
static motor_sentido_t sentido
    = MOTOR_DETENER;
```

Implementación — ControlAltura_Actualizar()

```
void ControlAltura_Actualizar(
    float altura_actual) {

    error_cm = objetivo_cm - altura_actual;

    if (fabsf(error_cm) < ZONA_MUERTA) {
        sentido      = MOTOR_DETENER;
        vel_aplicada = 0;

    } else {
        sentido = (error_cm > 0)
            ? MOTOR_SUBIR : MOTOR_BAJAR;

        float vel_calc = fabsf(error_cm) * KP;

        // Saturación superior
        vel_aplicada = (uint8_t)(
            (vel_calc > VEL_MAX)
            ? VEL_MAX : vel_calc);

        // Saturación inferior (vence inercia)
        vel_aplicada =
            (vel_aplicada < VEL_MIN)
            ? VEL_MIN : vel_aplicada;
    }

    Motor_Aplicar(sentido, vel_aplicada);
}
```

La única dependencia externa de height_control.c es Motor_Aplicar(). No incluye cabeceras de HAL, TIM ni GPIO. Esto permite pruebas unitarias con un stub de motor.

Consigna

Altura objetivo (cm)

3

Escalon a 5 cm

Escalon a 15 cm

Empuje

85

%

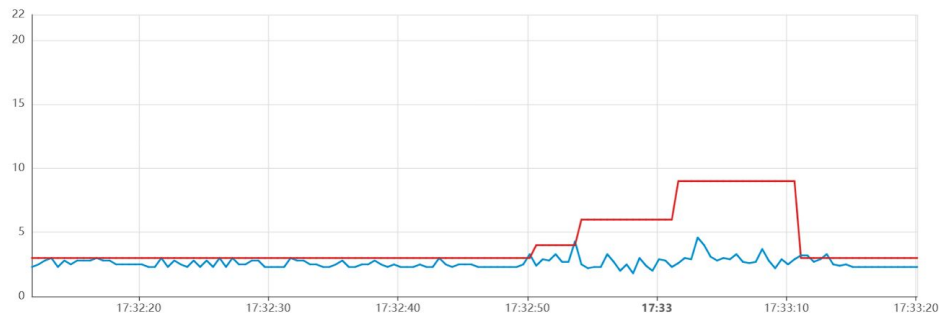
0

100

Respuesta al escalon

Altura vs Setpoint (cm)

Altura Setpoint



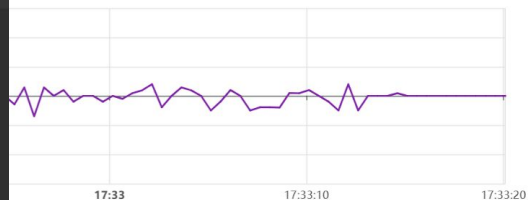
```
/* Ganancia proporcional: % de empuje por cada cm de error.
 * Parece chiquisima, y tiene que serlo: con la flotación al 60 %, un solo
 * 1 % de duty ya son ~16 cm/s² de aceleración. Ganancias del orden de las
 * unidades hacen que el conjunto se dispare. */
#define KP_PCT_POR_CM 1.0f

/* Ganancia integral: % de empuje por cada cm·s de error acumulado.
 * Corrige el desajuste de EMPUJE.BASE. Subirla acelera esa corrección pero
 * acerca el lazo a la inestabilidad: el criterio de Routh pide Kd·Kp > Ki. */
#define KI_PCT_POR_CM_S 0.05f

/* Ganancia derivativa: % de empuje por cada cm/s de velocidad vertical.
 * Es la que amortigua. Si el conjunto oscila, este es el valor a subir. */
#define KD_PCT_POR_CM_S 1.2f

/* Zona muerta sobre el término proporcional. En cero por defecto: con base
 * de empuje conviene corrección continua, y el promedio móvil ya deja el
 * ruido en torno a 0,1 cm. Subirla sólo si se ve temblar el empuje. */
#define ZONA_MUERTA_CM 0.0f

/* Saturación de la salida, alrededor de la base.
 * El mínimo no es cero: conviene que las hélices sigan girando para no
 * pagar el tiempo de arranque cada vez que hay que descender. */
```



Consigna



Escalon a 5 cm

Escalon a 15 cm

Empuje

95

%

0

100

Respuesta al escalon

Altura vs Setpoint (cm)

Altura Setpoint



Velocidad vertical (para ajustar Kd)

Velocidad vertical (cm/s)

Velocidad



```
/* Ganancia proporcional: % de empuje por cada cm de error.
 * Parece chiquisima, y tiene que serlo: con la flotación al 60 %, un solo
 * 1 % de duty ya son ~16 cm/s² de aceleración. Ganancias del orden de las
 * unidades hacen que el conjunto se dispare. */
#define KP_PCT_POR_CM 0.40f

/* Ganancia integral: % de empuje por cada cm*s de error acumulado.
 * Corrige el desajuste de EMPUJE_BASE. Subirla acelera esa corrección pero
 * acerca el lazo a la inestabilidad: el criterio de Routh pide Kd*Kp > Ki. */
#define KI_PCT_POR_CM_S 0.2f

/* Ganancia derivativa: % de empuje por cada cm/s de velocidad vertical.
 * Es la que amortigua. Si el conjunto oscila, este es el valor a subir. */
#define KD_PCT_POR_CM_S 0.9f
```

Consigna

Altura objetivo (cm)



Escalon a 5 cm

Escalon a 15 cm

Empuje

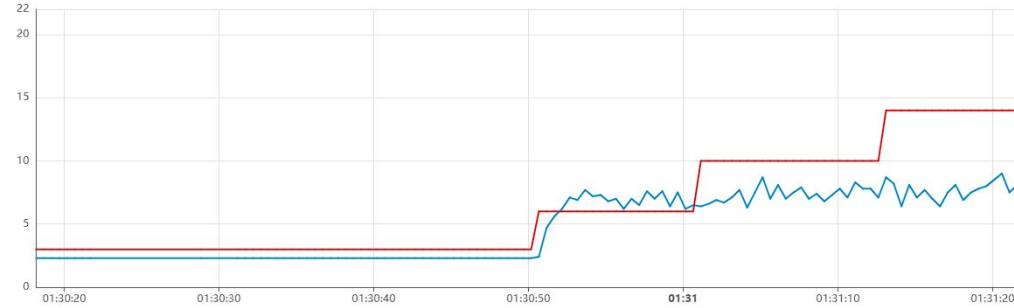
92
%

0

100

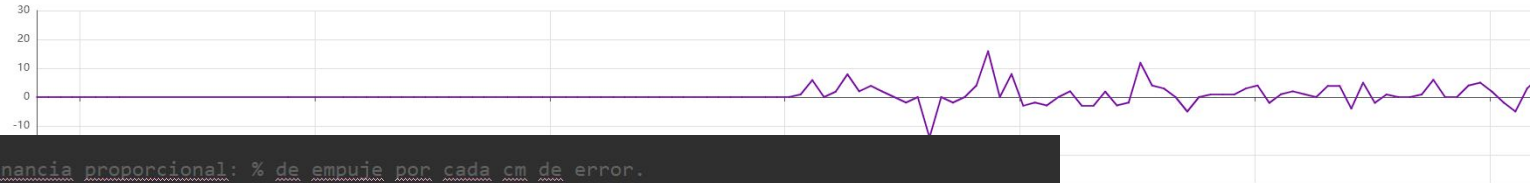
Respuesta al escalon

Altura vs Setpoint (cm)
—●— Altura —●— Setpoint



Velocidad vertical (para ajustar Kd)

Velocidad vertical (cm/s)
—●— Velocidad



```
/* Ganancia proporcional: % de empuje por cada cm de error.  
 * hice algunas pruebas y por ahora esto es lo mejor, podria hacer una simulacion */  
#define KP_PCT_POR_CM 0.40f  
  
/* Ganancia integral: % de empuje por cada cm*s de error acumulado. */  
#define KI_PCT_POR_CM_S 0.2f  
  
/* Ganancia derivativa: % de empuje por cada cm/s de velocidad vertical. */  
#define KD_PCT_POR_CM_S 0.9f  
  
/* Zona muerta sobre el término proporcional. lo fui probando  
 * hasta que dejo de temblar el empuje, le doy 0,5cm porque es  
 * el posible error */  
#define ZONA_MUERTA_CM 0.5f
```

Tabla comparativa: Proyecto original vs. Rediseñado

El salto cualitativo entre ambas versiones abarca organización del código, arquitectura de control, comunicaciones e interfaz de usuario. La tabla resume las siete dimensiones de comparación.

Aspecto	Proyecto original	Proyecto rediseñado
Organización del código	Único <code>main.c</code> monolítico, sin separación de responsabilidades	Módulos separados: <code>motor.c</code> , <code>distance.c</code> , <code>height_control.c</code> , <code>hc10.c</code> , <code>main.c</code> orquestador
Entrada del sistema	Potenciómetro analógico (ADC manual, PA4)	Setpoint remoto por Bluetooth desde dashboard Node-RED (<code>HEIGHT: <n>\n</code>)
Relación motor-sensor	Independientes, sin lazo. Distancia medida pero ignorada por el motor	Lazo cerrado de realimentación negativa con control proporcional ($K_p = 18\%/cm$)
Frecuencia de actualización	Sensor a ~ 1 Hz, sin control automático	Control a 10 Hz , telemetría a 2 Hz , desacoplados
Comunicación	Solo USART2 a 115200 baud (debug local, terminal serie)	USART2 (debug) + UART4/HC-05 Bluetooth (comando y telemetría remota, 9600 baud)
Interfaz de usuario	Ninguna (terminal serie cruda, lectura manual)	Dashboard Node-RED: slider, botones de escalón, gráficos de respuesta, gauge de empuje
Alcance de motores	1 motor / 1 canal L298N en firmware único	1 motor en firmware principal + variante experimental de 4 motores (<code>variante_4_motores</code>) aislada del código de producción

Modularidad

De 1 archivo a 5 módulos con responsabilidad única, testeables y reemplazables de forma independiente.

Control automático

De velocidad manual por potenciómetro a lazo cerrado proporcional con zona muerta y saturación.

Telemetría remota

De terminal serie local a protocolo Bluetooth bidireccional con dashboard gráfico en tiempo real.

Extensibilidad

Roadmap visible: término derivativo K_d , protocolo extendido VZ/S/L, variante de 4 motores aislada.